MedGlass -- Project Profile

Track 1: Development of Multimodal Content Creation Tools
AMD AI DevMaster Hackathon 2026

MedGlass is a fully-local, multimodal ambient clinical scribe: it listens to the consultation, looks at what the clinician looks at, and generates a structured SOAP note -- with every model running on a single AMD Radeon GPU. No audio, image, or patient text ever leaves the exam room.

---

1. Project background

Clinicians spend up to half of their working day on documentation. Ambient AI scribes promise to give that time back, but mainstream offerings are cloud services: the patient conversation -- the most sensitive data in the room -- is streamed to third-party servers. For many clinics and health systems that is a non-starter under HIPAA/GDPR, and rural or low-connectivity sites cannot use them at all.

MedGlass takes the opposite position: **privacy by architecture**. The entire multimodal pipeline -- speech-to-text, visual detection, medication-label OCR, and medical LLM note generation -- runs on one edge box with a single AMD Radeon AI PRO R9700 GPU. There is no cloud path in the codebase at all; "offline" is the only mode.

The content MedGlass creates is multimodal in and multimodal out:

- **In:** ambient audio (consultation speech) + vision (photos of medication bottles, rashes, forms -- detected and OCR'd).
- **Out:** a structured SOAP note with sections, the retained transcript, the case photos, spoken voice feedback (local TTS), and EHR-ready exports (HL7 FHIR R4 Bundle, CDA/CCD XML, Markdown/JSON).

2. Target users & application scenarios

Primary users

User	Need
Physicians / NPs / PAs	Hands-free documentation during the encounter; no typing, no "let me type this into the computer"
Clinics & health systems	HIPAA-compliant ambient scribing without cloud PHI exposure or per-encounter SaaS fees
Rural / field / military medicine	Full capability with zero internet connectivity
Medical scribes & informatics teams	Structured, exportable output (FHIR/CDA) instead of free text

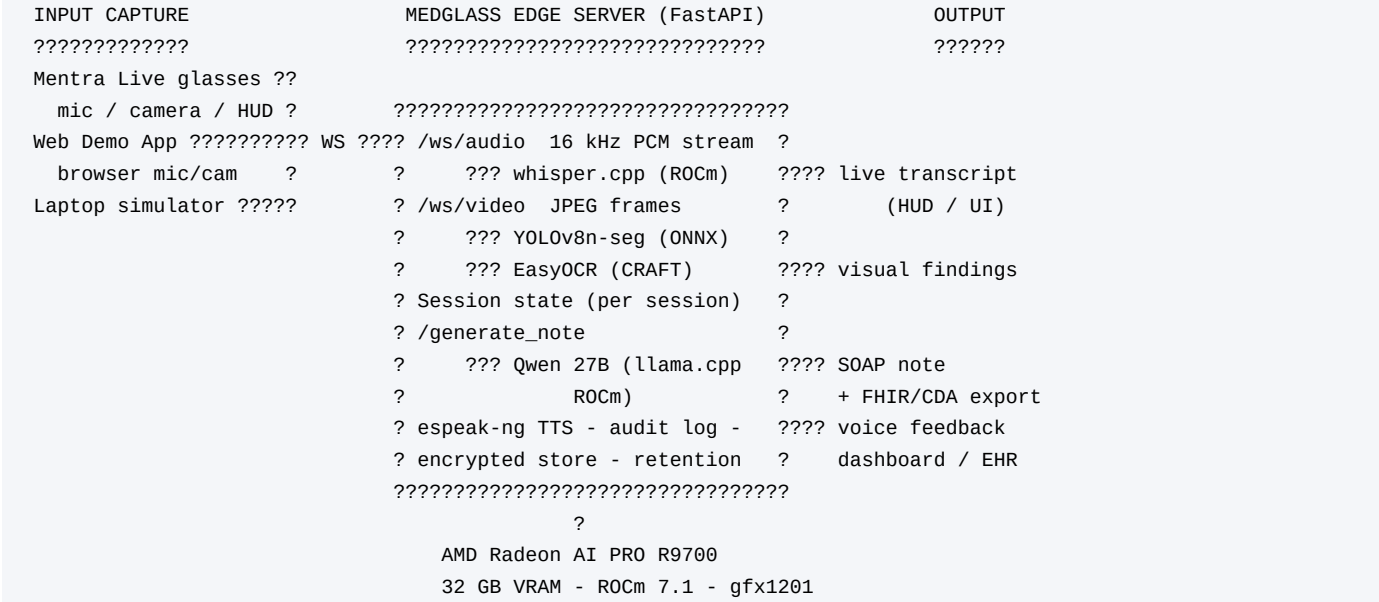
Application scenarios

1. **Standard ambulatory visit** -- the conversation is transcribed in real time; at the end of the visit a SOAP note is generated, reviewed on the dashboard, and exported to the EHR.
2. **Medication reconciliation** -- the clinician photographs the patient's medication bottles; YOLO detection + OCR reads the labels ("Ibuprofen 200 mg tablets (NSAID)") and the findings land in the note's Objective

section automatically.

3. **Dermatology / wound visual context** -- photos of the affected area are attached to the note with detection metadata.
4. **Care-team follow-through** -- the Plan section is parsed into next steps on the dashboard (assignable to Nurse/Provider, checkable off, with live follow-up / test / medication counters).
5. **Glasses-based workflow** -- with Mentra Live camera glasses: long-press to start, speak, short-press to capture, long-press to end; feedback via voice (TTS) and LED, results on the phone.

3. System architecture



Key components

- **Edge server** (src/server.py, FastAPI + uvicorn): WebSocket ingestion, session management, REST API, static web UI, security middleware.
- **Audio pipeline** (src/audio_handler.py): rolling 5 s window with 1 s overlap, 2-byte PCM alignment, deduplicated incremental transcription, end-of-session flush so the last words are never lost.
- **Vision pipeline** (src/video_handler.py): YOLOv8n-seg detection -> region-cropped EasyOCR for medication labels.
- **Note engine** (src/llm_handler.py, src/context_builder.py): SOAP prompt template with few-shot examples and PHI sanitizer; backends: llama.cpp (ROCm) -> Ollama -> deterministic template (auto-fallback).
- **Web UI** (src/static/): Patient Notes dashboard (search, S/O/A/P drawer, photos, transcript, FHIR/CDA/JSON/MD download, care-team next steps) and the Demo App (live mic recording with on-screen transcript, voice feedback, photo capture with preview).
- **Security & compliance layer**: scrypt password hashing, TOTP 2FA, login lockout, CSRF double-submit, session idle expiry, IP allow-list, audit log (JSONL), encryption at rest for notes/photos, retention purge, no-store caching for PHI pages. See docs/HIPAA_COMPLIANCE.md.

4. Model & algorithm introduction

Stage	Model	Format	Role
Speech-to-text	OpenAI Whisper small.en	GGML, Q5_1 via whisper.cpp	Ambient transcription of the consultation (English), ~109x real-time on GPU
Object detection	YOLOv8n-seg	ONNX (ONNX Runtime)	Detects medication bottles, documents, clinical objects in captured frames
OCR	EasyOCR (CRAFT + recognition net)	ONNX	Reads medication labels / text in detected regions ("Ibuprofen 200 mg tablets (NSAID)")
Note generation	Qwen 27B medical prompt	GGUF Q4_K_M via llama.cpp	Converts transcript + visual findings into a structured SOAP note
Voice feedback	espeak-ng	native	Local TTS announcements (session start/end, photo captured, note summary) -- no cloud TTS

Pipeline algorithm

1. Audio arrives as 16 kHz PCM over WebSocket; a rolling 5 s / 1 s-overlap window is transcribed at most once per second (serialized on a single GPU lock); incremental results are deduplicated against the previous window tail and streamed back to the client.

2. Photos are run through YOLO detection; medication-class boxes are cropped and OCR'd; findings are attached to the session.

3. On session end, any untranscribed audio tail is flushed (final whisper pass), then the context builder assembles transcript + visual findings into the SOAP prompt; the LLM generates the note, which is split into S/O/A/P sections, archived (encrypted), and exported on demand.

5. Adaptation description for AMD Radeon GPU / ROCm

Hardware / stack: AMD Radeon AI PRO R9700 (Navi 48, RDNA4, gfx1201), 32 GB VRAM, system ROCm 7.1, Ubuntu, amdgpu kernel driver.

What was adapted

- **whisper.cpp built with GGML_HIP=ON** (hipBLAS) targeting gfx1201 -- native HIP acceleration for transcription.
- **llama.cpp built with GGML_HIP=ON** targeting gfx1201 -- 100% of the 27 B model's layers offloaded to the GPU (-ngl 99), Q4_K_M quantization chosen to fit ~16.8 GB resident in the 32 GB framebuffer.
- **ONNX Runtime** for YOLOv8n-seg with automatic ROCm ExecutionProvider detection (falls back to CPU EP where ROCm Tracer is unavailable on Ubuntu-native ROCm -- documented, measured, and non-blocking since the two compute-heavy stages are fully GPU-native).
- **Quantization strategy:** Q5_1 for whisper (accuracy-critical, small), Q4_K_M for the LLM (quality/VRAM balance) -- both validated on the R9700.
- **VRAM budgeting & lazy loading** (memory_budget_gb in src/config.py):

models load on first use; concurrent residency whisper 1.5 GB + YOLO 0.5 GB + LLM 16.8 GB ~ 18.8 GB, comfortably inside 32 GB.

- **ROCm introspection:** rocm-smi VRAM traces, --showpids residency proof, GPU auto-detection (/dev/kfd, sysfs, lspci) surfaced live in /health and the benchmark harness.

Measured results (R9700, ROCm 7.1 -- benchmarks/cpu_vs_gpu.json)

Stage	CPU	GPU (ROCm)	Speed-up
Whisper small.en (5 s window)	6.96 s	1.10 s	**6.3x** (RTF 0.009 ~ 109x real-time)
Qwen 27B Q4_K_M note generation	25.7 s wall / 2.06 tok/s	7.2 s wall / 10.97 tok/s (peaks 28-47 tok/s)	**~3.6x wall / ~5.3x throughput**
YOLOv8n-seg	87.8 ms/frame	CPU EP (ROCm EP unavailable on this stack)	--
VRAM used during full pipeline	--	1.1 -> 18.4 GiB (rocm-smi trace)	fits 32 GiB card

Reproducibility: bash setup.sh installs dependencies, downloads models, and builds both HIP binaries when ROCm + a compiler are present; python3 benchmarks/benchmark.py regenerates the CPU-vs-GPU numbers.

Deliverables map

Requirement	Where
Project Profile (this document)	docs/PROJECT_PROFILE.md (+ PDF)
Source code repository	repo root (src/, client/, scripts/, benchmarks/, tests/)
README: env config, startup, dependencies	README.md, requirements.txt, setup.sh, .env.example
Demo video (3-5 min)	demo/medglass_demo.mp4
GPU evidence	benchmarks/cpu_vs_gpu.json, benchmarks/gpu_utilization.png, demo/evidence_*.png